

COMP101

Introduction to Programming

2025-26

Lecture-24

2-d list ('2-d array, 2-d matrix')

Strings and Numbers

Multidimensional Structures

- list, array, matrix
 - 1-dimensional, 2-dimensional
- store data in elements
- elements have similar properties
- Python has no built-in support for arrays
 - use a list
 - Full array handling is via library import NumPy

Multidimensional Structures

- list – generalised form of an array
 - ordered, multiple data types
 - generalised form of array
- array – specialised form of a list
 - ordered, same data types
 - generalised form of a matrix
- matrix - specialised form of an array
 - multi-dimensional for maths and science

<https://www.freecodecamp.org/news/python-array-tutorial-define-index-methods/>

One-dimensional ('1-d') structure

- 1-d structures ('1-d list', '1-d array' '1-d matrix')
 - visualise as a set of rows and a set of columns



- list dimension = 1×7
 - one row, seven columns
 - Index [0] to [6] gives 7 'elements'

Two-dimensional ('2-d') structure

- 2-d structures ('2-d list', '2-d array' '2-d matrix')
 - visualise as a set of rows and a set of columns

- list dimension = 4 x 7
 - four rows and seven columns
- always read in 'row-column order'

Two-dimensional ('2-d') structure

- 2-d list 4x7

- Intersection of a row and a column is a 'cell'
- A cell is an 'element'
 - This 2-d list has 28 elements

Two-dimensional ('2-d') structure

r/c	[0]	[1]	[2]	[3]	[4]	[5]	[6]
[0]	0,0	0,1	0,2	0,3	0,4	0,5	0,6
[1]	1,0	1,1	1,2	1,3	1,4	1,5	1,6
[2]	2,0	2,1	2,2	2,3	2,4	2,5	2,6
[3]	3,0	3,1	3,2	3,3	3,4	3,5	3,6

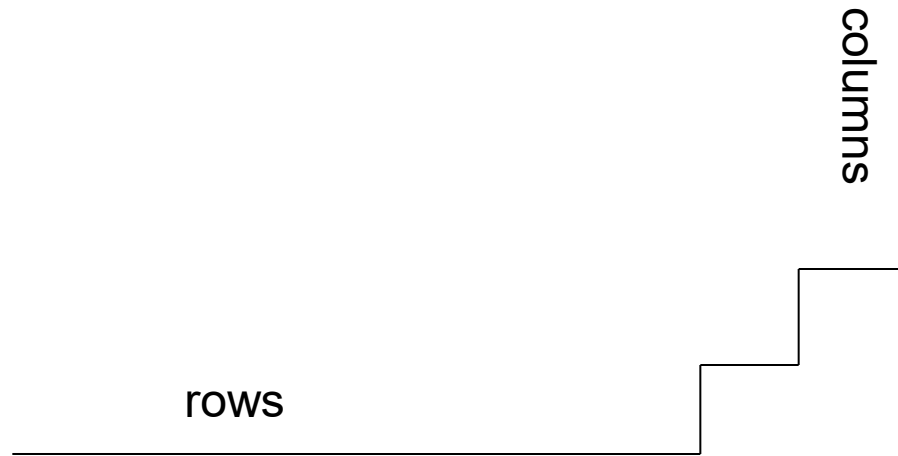
- Each row is indexed, each column is indexed
- Elements have two index values
 - First index for row, second index for column

Two-dimensional ('2-d') structure

r/c	[0]	[1]	[2]	[3]	[4]	[5]	[6]
[0]	0,0	0,1	0,2	0,3	0,4	0,5	0,6
[1]	1,0	1,1	1,2	1,3	1,4	1,5	1,6
[2]	2,0	2,1	2,2	2,3	2,4	2,5	2,6
[3]	3,0	3,1	3,2	3,3	3,4	3,5	3,6

- first element = [0,0] = row[0], column[0]
- last element = [3,6] = row[3], column[6]
- Access to an element is 'row-column order'
 - The row index always comes first

2-d addressing



‘run’ (r) along corridor before we

‘climb’ (c) the stairs

‘row–column order’: row index comes first then column index:

The dimension of a 2-d list is (row x column)

Often written as (r x c) or (m x n)

2-d addressing

“dimension = 3 x 4” means 3 rows, 4 columns

	[0]	[1]	[2]	[3]
[0]	0,0	0,1	0,2	0,3
[1]	1,0	1,1	1,2	1,3
[2]	2,0	2,1	2,2	2,3

Keep the orientation this way:

First element always in top left-hand corner, last element always bottom right-hand corner. Array dimension 3x4 = 12 elements

1-d vs 2-d

1-d list has data in linear order

`meal_list = [soup, salad, fish, chips, peas, beans, cake, jelly]`

soup	salad	fish	chips	peas	beans	cake	jelly
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]

`num_list = [1, 2, 3, 4, 5, 6]`

1	2	3	4	5	6
[0]	[1]	[2]	[3]	[4]	[5]

Arrays contain same data types (lists can be mixed type)

1-d vs 2-d

2-d list has data as a 'list of lists'

```
meal_list= [ [soup, salad], [fish, chips, peas, beans], [cake, jelly] ]
```

row index: [0] [1] [2]

Outer brackets define a 1-d list:

```
meal_list = []
```

Inner brackets define the nested lists – the rows:

```
meal_list = [ [], [], [] ]
```

- three rows in this 2-d list

1-d vs 2-d

Dimension is determined by the row with the most values in it:
dimension = 3 x 4 because meal_list [1] has four values in it

```
meal_list= [ [soup, salad], [fish, chips, peas, beans], [cake, jelly] ]
```

soup	salad		
fish	chips	peas	Beans
cake	jelly		

Some elements are empty – these are ‘null elements’
The rest of the elements are ‘populated elements’

1-d vs 2-d

dimension = (2 x 3)

num_list = [[0,3,5], [2,4,6]]

0	3	5
2	4	6

- All six elements are 'populated' with no null (empty) elements
- num_list [0,0] is populated with value = 0
 - There is a value in it - it is not a null element – the value is zero
 - If it had no value in it (i.e. empty) it would be a 'null element'

List vs Array

- List
 - stores data which is indexed
 - can be iterated through
 - elements can be of different type
 - can use as an array but with no specialised properties and tools for scientific computing

List vs Array

- Array
 - stores data which is indexed
 - can be iterated through
 - elements are of same type
 - has specialised properties and tools for scientific computing
 - Can access different functions compared to a list
 - e.g. divide array by 3 means divide all elements by 3

NumPy and array type (Not used in COMP101)

- Python has array type as well as a list type

`import array`

- Usually used for arithmetic
- Looks like a list but with elements all same type
 - primitive types (numeric or bytes)

- ‘NumPy’

- NumPy - the core library for scientific computing
- usually for matrices

`from numpy import *`

http://www.physics.nyu.edu/pine/pymanual/html/chap3/chap3_arrays.html